Lab 5: Forecasting Connecticut's Electricity in Production · Tonight in 20 Steps

OPIM 5512 · Module 4 · Solo · Colab · 33,215 hours of Bradley weather + Connecticut load, then your own energy project

> **The one-sentence version: tune a real time-series model, deploy it to Google Cloud, and watch it forecast the next 24 hours, every hour.** You find the rhythms, build features production can use, beat three baselines, tune three model families without peeking at the future, ship your champion, and see your first real forecast next to ISO New England's.

Before we start

- [ ] **Pre-lab done:** four green deploys on your energy fork (setup guide, Step 9), and `scraper-isone` says `"status": "ok"`
- [ ] You know your **energy project ID** and **bucket name** (usually the same)
- [ ] The notebook is open: HuskyCT → In-Class Labs → **Lab 5**

> ⚠️ **The 3 things that trip everyone up (read this first)**
>
> 1. **Save your work:** the notebook opens from GitHub, so first do **File → Save a copy in Drive**.
> 2. **Never shuffle a time series.** Train on the past, test on the future (Part 4 does it for you). A shuffled split looks great in Colab and fails in production.
> 3. **The Google pop-up in 7b: pick the Gmail you used for your ENERGY project.** Wrong account = "cannot see the bucket".

The 20 steps, in order

Part 0–1 · Start up and load (steps 1–3, 10 min)

1. Open the notebook from HuskyCT → **File → Save a copy in Drive** (work in the copy). **Part 0:** four green deploys? Good.
2. **1a:** run the install cell (scikit-learn **1.6.1**, the same as production). Asked to restart? Restart, then run from the top.
3. **1b:** type your **energy** `PROJECT_ID` and `BUCKET_NAME` → load the data → **33,215 hours, Jan 2023 to Oct 15, 2026**.

Part 2 · Look (steps 4–6, 15 min)

4. **2a–2b:** the **daily** rhythm (low at 3–4 a.m., peak late afternoon; weekends lower) and the **yearly** one (summer peak, smaller winter peak).
5. **2c:** the **U-shape**: hot hours (air conditioning) and cold hours (heat, lights) use the most. Temperature is the star.
6. **2d: yesterday predicts today**: load vs 1 hour ago **0.97**, same hour yesterday **0.88**, same hour last week **0.74**.

Part 3 · Features (step 7, 10 min)

7. Run `make_features` (calendar + 5 weather columns + **load at the same hour yesterday**) → **33,191 hours, 11 features**. This exact function runs inside `forecast-24h`: change it here and production stops agreeing.

Part 4 · Baselines (steps 8–9, 10 min)

8. **4a:** split by **time**: **26,280 training hours (2023–2025)** / **6,911 test hours (2026 so far)**.
9. **4b:** the numbers to beat (MAE in MW): **always the average 579**, **same hour last week 404**, **same hour yesterday 266**.

Part 5 · Tune (steps 10–13, 25 min)

10. **5a:** `TimeSeriesSplit`: 5 folds, each trains on the past and validates on the six months right after it.
11. **5b:** run the grid search (about a minute): decision tree **179**, random forest **144**, gradient boosting **121** (`train_cv_mae_mw`).

12. **Class challenge:** add values to a grid, re-run, try for the lowest `train_cv_mae_mw`. **5c:** your champion → test MAE about **120 MW** (3.9 %).
13. **5d:** the **loss curve**: watch boosting learn, tree by tree. Where does the validation error flatten?

Part 6 · Look at the misses (steps 14–16, 15 min)

14. **6a–6b:** this summer's hottest week and a January week, then error by hour of the day + the 5 worst hours of 2026.
15. **6c–6d:** permutation importance (boxplot) and partial dependence: did your model learn the U-shape and the daily rhythm?
16. **Your turn (you write the code):** add `load_last_week_mw`, refit, compare test MAE. Why can't production use it yet?

Part 7 · Ship it (steps 17–18, 15 min)

17. **7a:** save `model.joblib` + `model_card.json`. **7b:** sign in (pick your **energy** Gmail) → 3 checks: account, bucket, `forecast-24h` ACTIVE.
18. **7c:** upload. **7d:** run `forecast-24h` now → dots → **"Done … energy/forecasts/latest.csv"**.

Part 8–9 · Tomorrow and wrap up (steps 19–20, 10 min)

19. **Part 8:** your **first real forecast**: the next 24 hours, the last 48 hours of actual load, and ISO New England's forecast.
20. Answer the **five short questions** → **File** → **Download** → **.ipynb** → upload to **Submit Lab 5** (due **Thu Nov 12, 11:59 PM**).

Your forecast files (a new forecast every hour at :20)

File	What is in it
`gs://YOUR-BUCKET/energy/forecasts/latest.csv`	your next 24 hours: `target_hour_utc`, `predicted_load_mw` (Lab 6 reads this one)
`gs://YOUR-BUCKET/energy/datasets/forecasts_master.csv`	every forecast you ever made, tagged with the model version

If you get stuck

You see	Do this
A missing variable (`NameError`)	Run every cell above it first (Runtime → Run before)
7b: "cannot see the bucket"	Runtime → Disconnect and delete runtime, run again, pick your **energy** Gmail
7b: "forecast-24h: NOT FOUND"	Setup guide Step 9: run **Deploy forecast-24h** on your fork, wait for green
7d log says `"no_load_data"`	ISO secrets missing/misspelled: fix them, re-run **Deploy scraper-isone**
7d: "No new forecast after 4 minutes"	Run the log cell under 7d and paste the log into Claude
Part 8: "DEMO"	Run 7d first, then re-run Part 8

Leave everything running. Every hour your project collects weather and load and makes a new forecast: that's Lab 6.